Design Pattern Identification and Evolution in an ASP.NET Core Vehicle Rental Application

23CSE455 Design Patterns · Case Study · Team B6

Samridhi Singh · Bhavana P H · Neema Vinod · Vani Sugovind S R

Amrita School of Computing, Amritapuri · October 2026

Repository · Dependency Injection · Service Layer · Middleware Chain · Producer–Consumer · Adapter · Proxy · Strangler Fig · DTO · State · Factory Method · Strategy · Observer

Team B6 and the case study

Samridhi Singh

AM.SC.U4CSE23251

Change1 · Fleet-maintenance scheduling

State

Bhavana P H

AM.SC.U4CSE23211

Change2 · New vehicle categories

Factory Method

Neema Vinod

AM.SC.U4CSE23238

Change3 · Interchangeable pricing

Strategy

Vani Sugovind S R

AM.SC.U4CSE23266

Change4 · Booking notifications

Observer

Assigned case study

DOMAIN

Vehicle rental and fleet management

FRAMEWORK

ASP.NET Core 8 Web API, EF Core 8, MySQL

APPLICATION

WanderCar / Fleman — open-source FleetManagment repository on GitHub

OUR SCOPE

The .NET backend (FlemanApi) and its NUnit test project

SUBMISSION

23CSE455_DP_CaseStudy / B6

The case study at a glance

9 → 13

pattern instances, original code to final code

4

realistic changes, each adding one GoF pattern

183 → 297

tests; the final code passes all 297

4 × 5

languages × patterns re-implemented in Part 3

1 Part 1 · Discovery

LLM-assisted scan of the original code, every candidate checked against source: 9 confirmed, 3 uncertain, 9 rejected.

2 Part 2 · Evolution

State, Factory Method, Strategy and Observer introduced by four changes; two real refactorings; counts and UML updated per version.

3 Part 3 · Cross-language

Same problem, same tests, in Java, Python, JavaScript and C++; LOC, cyclomatic complexity and run time compared.

01

Application and architecture

WanderCar / Fleman — the .NET backend of an open-source vehicle rental system

Application, source and scope

Item	Value
Source	github.com/Naveen-verma-7697/FleetManagment, branch main
Baseline commit	dfc61ab8 (2026-08-24) “Clean up comments in IAvailabilityRepository”
Analysed	WanderCar_DotNet/src/FlemanApi and FlemanApi.Tests
Stack	ASP.NET Core 8, EF Core 8 + Pomelo MySQL, AutoMapper, FluentValidation, MailKit, QuestPDF
Tests	NUnit 3.14, Moq 4.20, FluentAssertions 6.12, EF Core InMemory 8.0.2
Environment	Windows 11 Pro, .NET SDK 8.0.425 / runtime 8.0.31, PlantUML 1.2024.8

Inspected

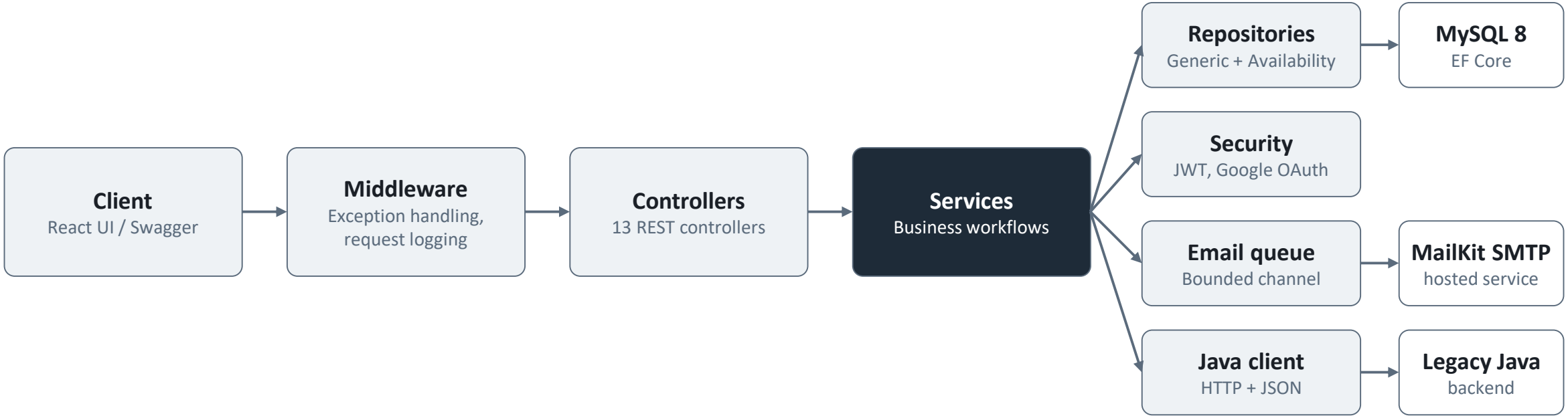
Controllers · Service · Repository · Data · Models · DTO · Middleware · Security · Util · Validators · AutoMapper · Program.cs

All eight functional areas of the brief: vehicles, customers, booking, pricing, billing, maintenance, notification, authentication.

Excluded

React frontend and the Java backend — not part of the .NET application; the Java service is reached only through an HTTP client.

Layered architecture and a request flow



Example flow — create a booking (original code)

1. POST /api/bookings passes the logging and exception middleware
2. BookingController calls IBookingService
3. BookingService validates, checks capacity and prices with RentalRateCalculator
4. GenericRepository saves through FlemanDbContext to MySQL
5. Confirmation e-mail is queued; a hosted service sends it via MailKit

Baseline build and test run

183
tests in the original code

182
passed

1
failed — date-dependent, not a bug we introduced

Build
0 warnings
0 errors

Why one test fails

StaffServiceTests.ProcessReturnAsync_CreatesInvoiceHeaderAndDetails

The fixture hands the car over on 2026-08-01, but StaffService bills up to DateTime.Now. Expected 40.0 (4 days), got 690.0 on the day we ran it.

This hidden clock dependency is a code smell we record here and fix in Change3.

Reproduce

```
cd B6\data\code\WanderCar_DotNet
dotnet test src\FlemanApi.Tests\
  FlemanApi.Tests.csproj
```

Build the test project, not DotnetBackend.slnx — .slnx needs SDK 9.0.200+ and fails on SDK 8 with MSB4068.

02

Part 1 · Pattern discovery

Which patterns does the original code really contain — with source evidence for every claim

Discovery method: LLM proposes, source decides

1 Fix the scope

Baseline commit dfc61ab8; file list in inspected_modules.txt

2 LLM scan

GPT-5.6 Luna on the extracted source context (2026-10-08); exact prompt saved

3 Check in code

Roles, interfaces, calls and delegation traced in the real files

4 Classify

Confirmed, uncertain or rejected — each with a written reason

9

confirmed — one instance per pattern type

3

uncertain — structure present, intent not provable

9

rejected — framework APIs or naming only

Pattern inventory of the original code

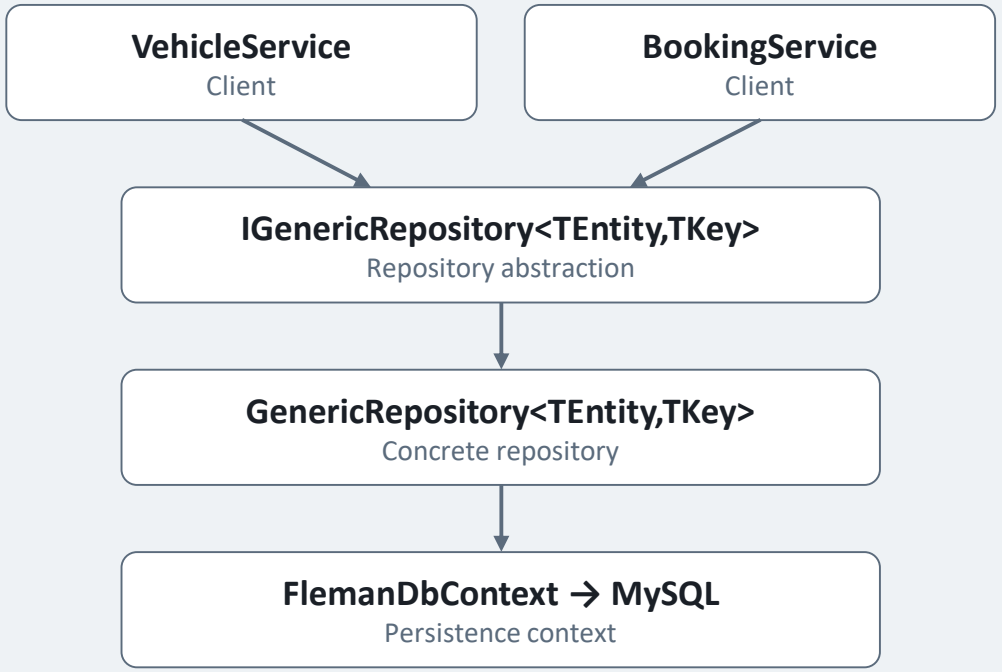
#	Pattern	Category	Instance	Key participants
1	Repository	Enterprise	Generic Repository	IGenericRepository, GenericRepository, FlemanDbContext; clients VehicleService, BookingService
2	Dependency Injection / IoC	Architectural	ASP.NET Core DI Composition	Interface → implementation bindings in Program.cs; constructor-injected services and controllers
3	Service Layer	Enterprise	Booking Service Layer	BookingController → IBookingService → BookingService
4	Middleware Pipeline / Chain of Resp.	Architectural	Request Middleware Chain	ExceptionHandlingMiddleware, RequestLoggingMiddleware, each calling _next(context)
5	Producer–Consumer	Concurrency	Email Background Work Queue	EmailService → EmailBackgroundQueue (bounded Channel) → EmailQueueHostedService → MailKitEmailSender
6	Adapter	Structural	Java Microservice Adapter	IJavaMicroserviceClient, JavaMicroserviceClient (HttpClient, Java JSON/DateTime)
7	Proxy	Structural	Legacy API Proxy	LegacyProxyController forwards /api/legacy through IJavaMicroserviceClient
8	Strangler Fig	Modernisation	Legacy Java Integration	JavaMicroserviceClient, JavaInvoicePdfService, LegacyProxyController, Program.cs
9	DTO	Enterprise	API Request/Response DTO Boundary	CarDTO, BookingResponseDTO, CreateBookingRequest …; services map entities to DTOs

Totals at baseline: 9 distinct pattern types · 9 instances · source: Part1/part1_patterns.csv

Two instance records in detail

Repository

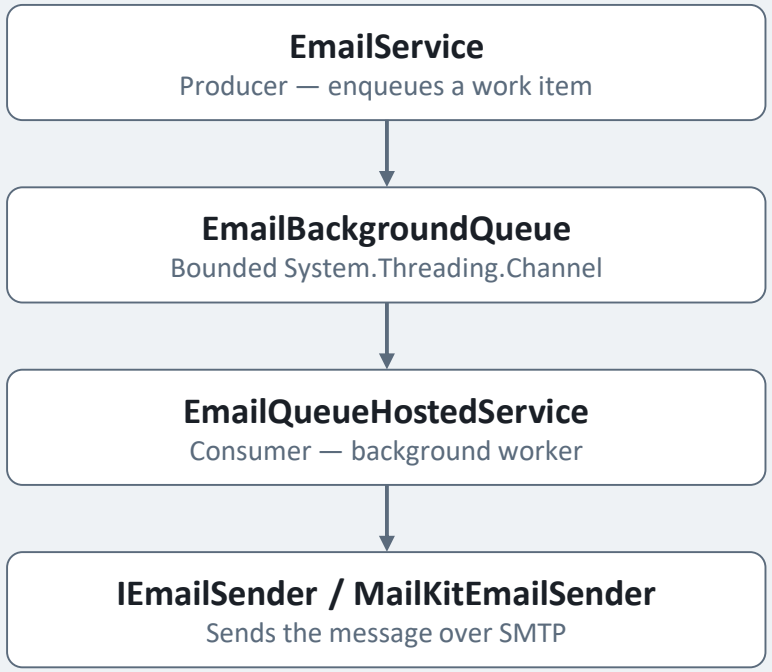
Generic Repository



Services see only the interface; CRUD and query logic over DbSet lives in one generic class.

Producer–Consumer

Email Background Work Queue



HTTP requests return without waiting for SMTP; the bounded channel throttles producers if the worker falls behind.

The other seven confirmed instances

DI / IoC

ASP.NET Core DI Composition

Program.cs binds every interface to an implementation; services and controllers receive collaborators through constructors.

Service Layer

Booking Service Layer

BookingController only delegates; BookingService owns the booking workflow and coordinates repositories.

Chain of Responsibility

Request Middleware Chain

Each middleware receives the next RequestDelegate and decides whether and when to call _next(context).

Adapter

Java Microservice Adapter

JavaMicroserviceClient hides HttpClient and Java-specific JSON and DateTime handling behind IJavaMicroserviceClient.

Proxy

Legacy API Proxy

LegacyProxyController exposes /api/legacy and forwards each call to the remote Java service.

Strangler Fig

Legacy Java Integration

Source describes the Java link as a strangler integration: invoices still come from Java while .NET takes over.

The ninth, the DTO boundary, separates request/response classes in DTO/ from the EF Core entities in Models/.

Rejected and uncertain candidates

Rejected	Reason
Factory	Only framework calls (CreateBuilder, AddScoped); no app-authored factory
Singleton	AddSingleton is a DI lifetime, not a private-constructor Singleton
Builder	Fluent WebApplication builder is framework API usage
Facade	EmailService simplifies, but no deliberate subsystem facade
Decorator	Middleware wraps a pipeline, not a component interface
Observer	OnCreatingTicket etc. are framework auth hooks
Command	Queued Func delegates, but no command abstraction or invoker
Template Method	No base class with an invariant algorithm and hook steps
State	BookingStatus / CarStatus are enums with if-rules, not state objects

Uncertain · Strategy

IInvoicePdfService has a second implementation, but it is commented out — no runtime choice.

Uncertain · Data Mapper

Mapping is done by AutoMapper (framework); DTO is confirmed instead.

Uncertain · Unit of Work

EF Core DbContext coordinates saves; no IUnitOfWork in the app.

State, Observer, Factory and Strategy were missing or doubtful here — Part 2 adds each of them for real.

03

Part 2 · Evolution and refactoring

Four realistic changes, each introducing one pattern — applied in order to the original code

Four changes, one new pattern each

CHANGE1

Fleet-maintenance scheduling

Samridhi Singh

State

+50 tests

No refactoring (additive)

CHANGE2

New vehicle categories

Bhavana P H

Factory Method

+24 tests

No refactoring (0 lines removed)

CHANGE3

Interchangeable pricing policies

Neema Vinod

Strategy

+22 tests

Static calculator → policy objects; injected clock

CHANGE4

Booking lifecycle notifications

Vani Sugovind S R

Observer

+18 tests

Extract Class + Move Method on BookingService

Integrated in order → Part2/code

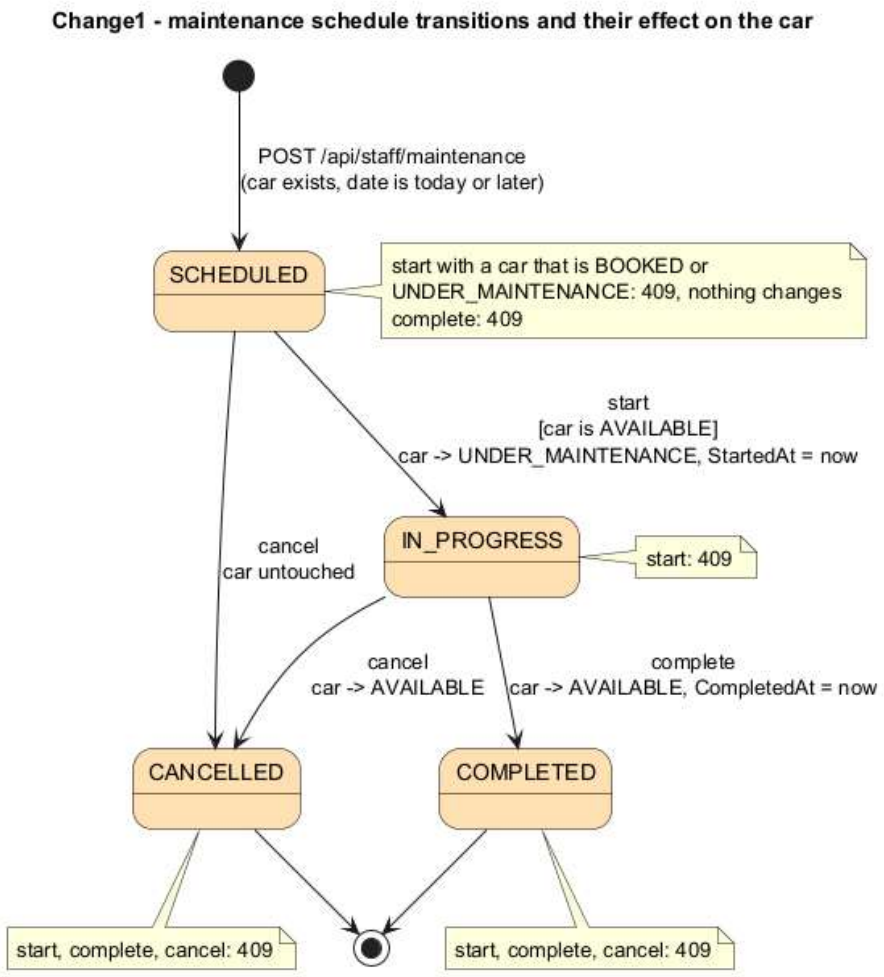
Original	After C1	After C2	After C3	After C4
182 / 183 →	232 / 233 →	256 / 257 →	279 / 279 →	297 / 297

Change1 · Maintenance workflow with State

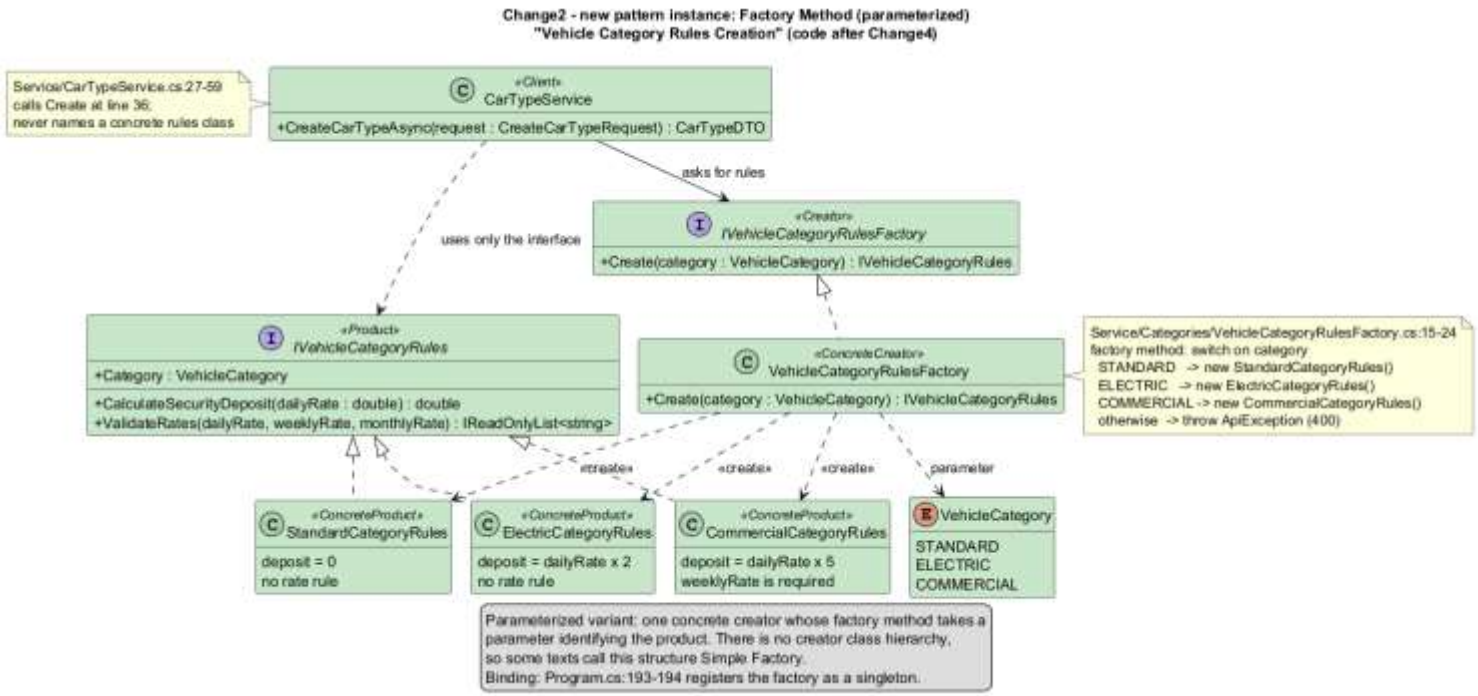
Staff schedule, start, complete or cancel maintenance for a car and list what is due or overdue. Each status is a class, so MaintenanceService has no switch on Status.

Role	Participant
State	IMaintenanceState — Start, Complete, Cancel
Default behaviour	MaintenanceStateBase — every operation → 409
ConcreteState	ScheduledState, InProgressState
Terminal states	CompletedState, CancelledState (override nothing)
Context	MaintenanceContext — picks state by dictionary lookup
Client	MaintenanceService.TransitionAsync

Result: +50 tests, 233 total / 232 passed (same baseline failure). New code takes time from an injected TimeProvider. Code in Part2/code is reconstructed from the change documents.



Change2 · Vehicle categories with Factory Method



CarTypeService asks the factory for the rules of a category and only ever uses IVehicleCategoryRules. The switch in Create is the single place that names concrete classes.

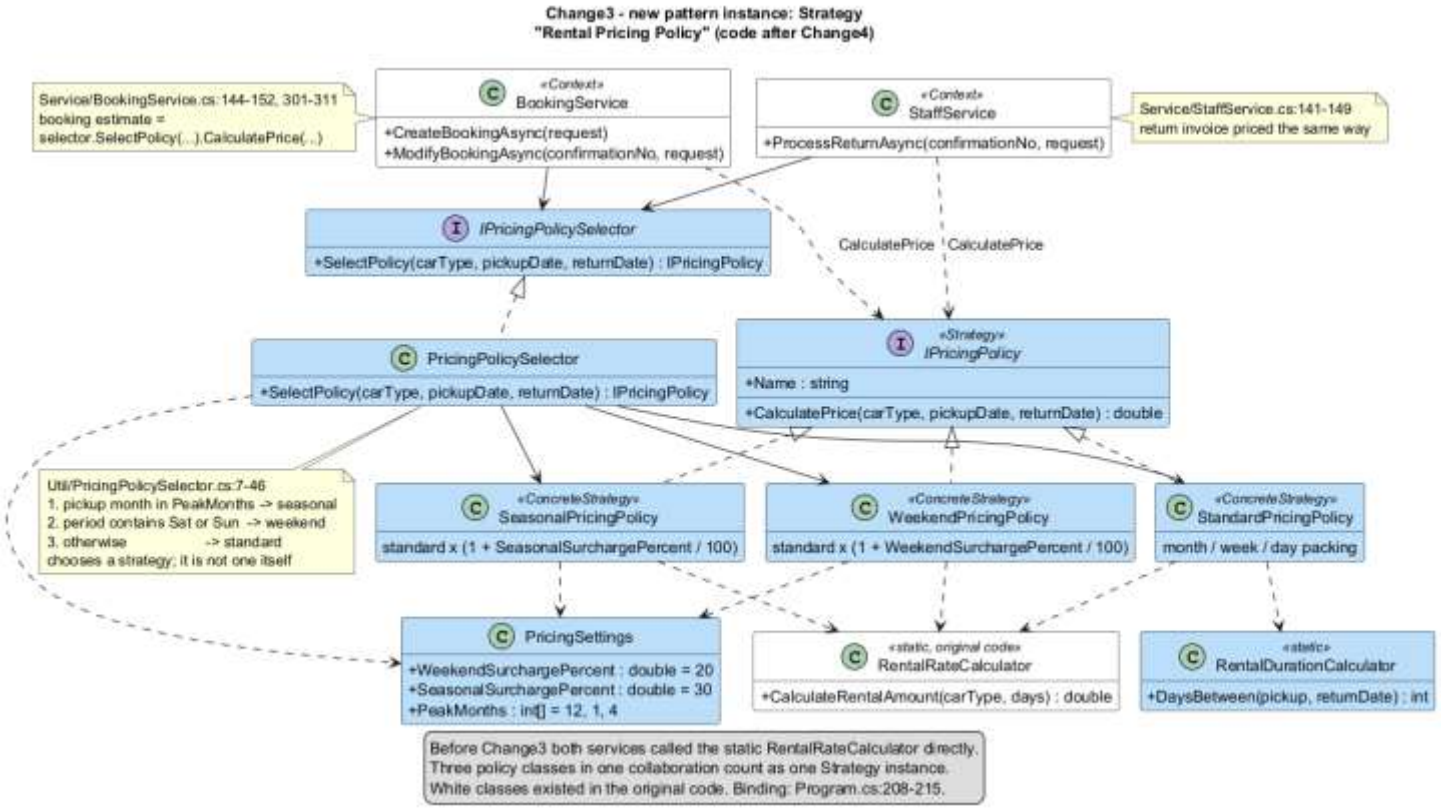
Parameterized Factory Method — one concrete creator; some texts call this Simple Factory.

Category	Deposit	Rate rule
STANDARD	0	none
ELECTRIC	2 × daily	none
COMMERCIAL	5 × daily	weekly rate required

Acceptance — all met

- Existing car types report STANDARD, deposit 0
- POST /api/staff/car-types stores 0 / 120 / 400 at run time
- COMMERCIAL without weekly rate → 400; duplicate → 409
- +24 tests; 0 existing lines removed

Change3 · Pricing policies with Strategy



Selector precedence: pickup in a peak month → seasonal (+30%); any Saturday or Sunday → weekend (+20%); otherwise standard. The selector chooses a strategy; it is not one.

Smell

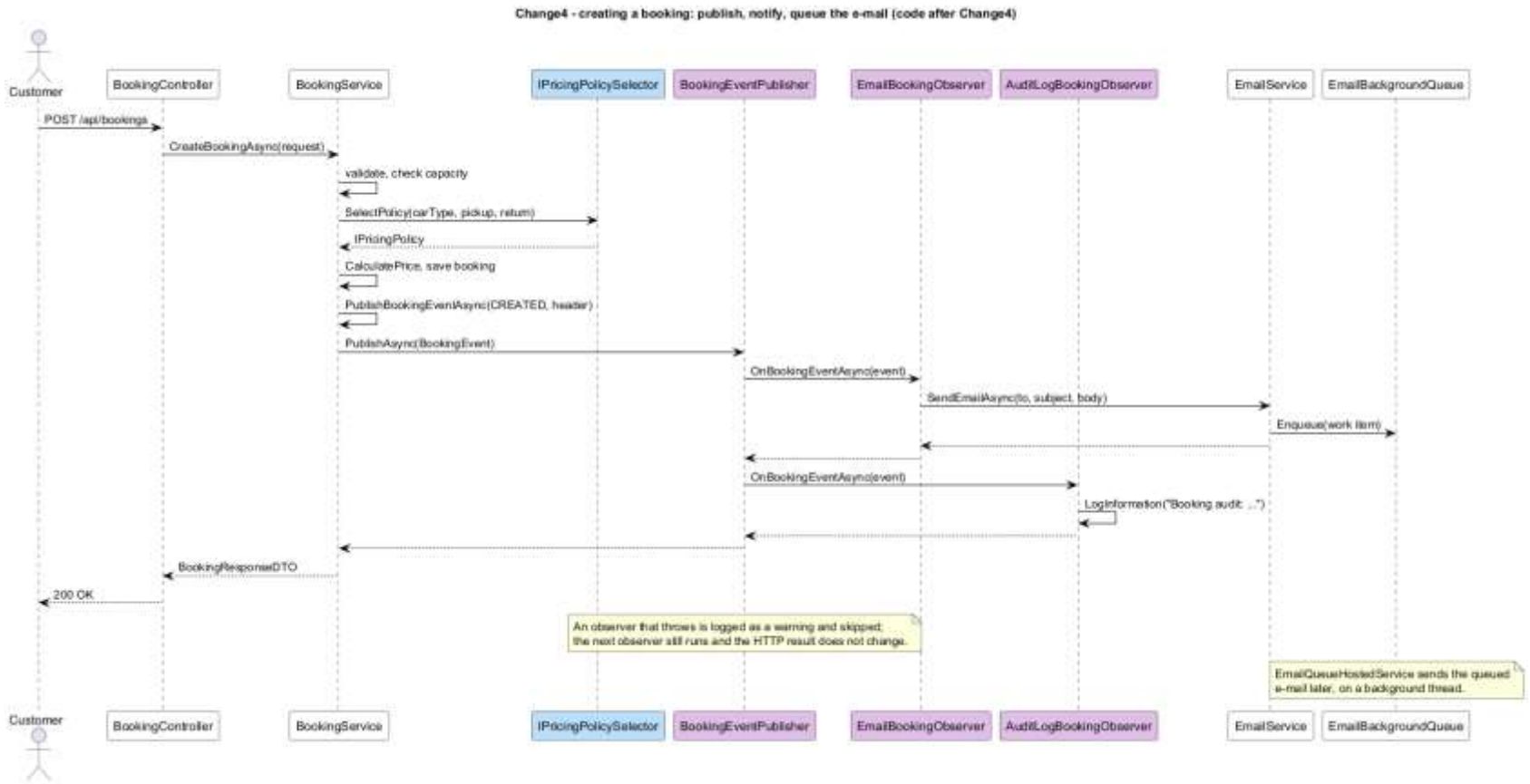
BookingService and StaffService called the static RentalRateCalculator directly — pricing could not vary without editing both services.

Refactoring

- Extract interface IPricingPolicy; three policies
- Standard policy delegates to the old calculator, preserving behaviour
- Inject TimeProvider into StaffService — the clock-dependent test now passes

+22 tests · 279 / 279 after this step

Change4 · Booking notifications with Observer



Push model: BookingService publishes one immutable BookingEvent after each successful create, modify or cancel. A failing observer is logged and skipped; the next one still runs.

Smell

Long Method: the 123-line CreateBookingAsync ended with 30 lines that built and sent the confirmation e-mail.

Refactoring

Extract Class + Move Method into EmailBookingObserver (same e-mail text), then replace the direct call with PublishBookingEventAsync.

+18 tests; all 21 old BookingServiceTests pass, only their SetUp changed

Effect of each change on every instance

Instance	Original	Change1	Change2	Change3	Change4
Generic Repository	baseline	**extended**	**extended**	preserved	preserved
ASP.NET Core DI Composition	baseline	**extended**	**extended**	**extended**	**extended**
Booking Service Layer	baseline	preserved	preserved	**modified**	**modified**
Request Middleware Chain	baseline	preserved	preserved	preserved	preserved
Email Background Work Queue	baseline	preserved	preserved	preserved	preserved
Java Microservice Adapter	baseline	preserved	preserved	preserved	preserved
Legacy API Proxy	baseline	preserved	preserved	preserved	preserved
Legacy Java Integration	baseline	preserved	preserved	preserved	preserved
API Request/Response DTO Boundary	baseline	**extended**	**modified**	preserved	preserved
Maintenance Schedule Workflow (State)	—	**new**	preserved	preserved	preserved
Vehicle Category Rules Creation (Factory)	—	—	**new**	preserved	preserved
Rental Pricing Policy (Strategy)	—	—	—	**new**	preserved
Booking Lifecycle Notification (Observer)	—	—	—	—	**new**

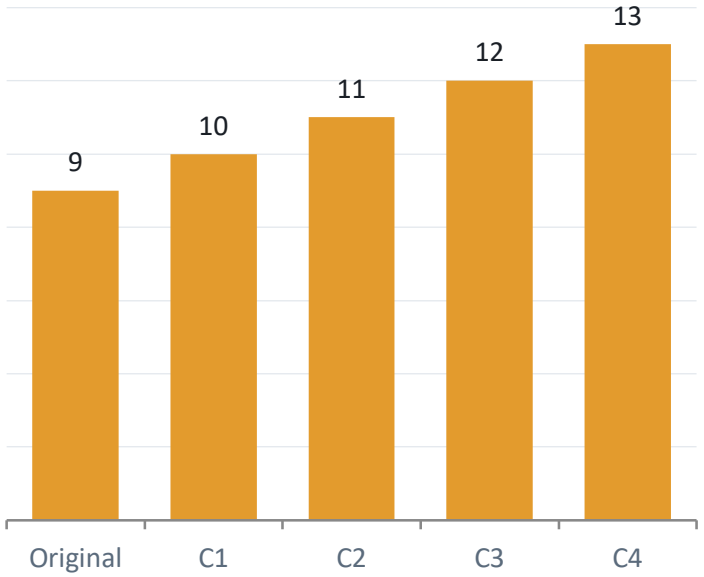
No instance removed or replaced
Counts rise 9 → 10 → 11 → 12 → 13.

DI absorbs every change
Each change only adds registrations.

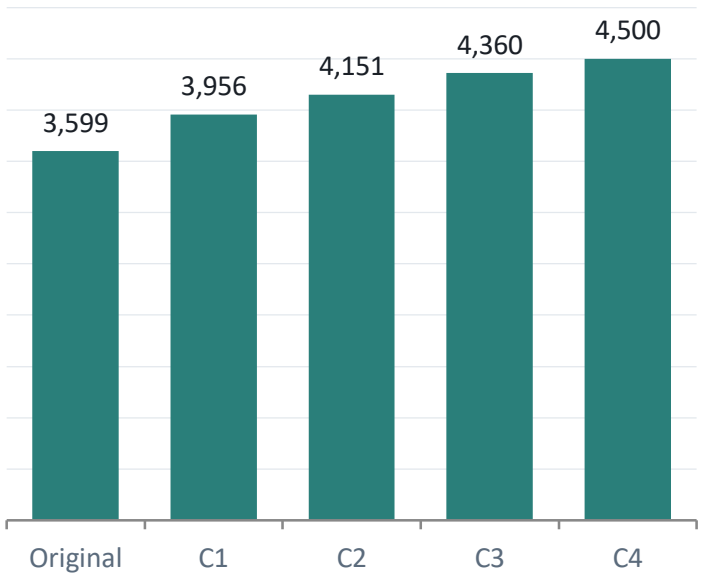
Booking Service Layer modified twice
Pricing selector (C3), event publisher instead of IEmailService (C4); IBookingService unchanged.

Before and after metrics, per version

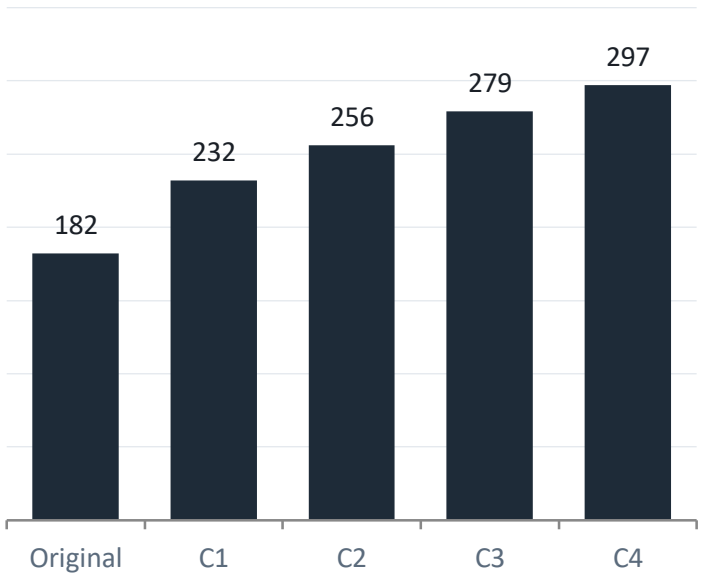
Pattern instances



Source LOC (FlemanApi)



Tests passed



Same rule every version: non-blank, non-comment .cs lines in FlemanApi, excluding Migrations, bin and obj; tests from dotnet test on SDK 8.0.425. Original → final: 67 files changed (58 added, 9 modified), +4,545 / −52 lines including tests and migrations. Complexity, coverage and coupling were not measured for the C# code, and more LOC alone does not mean better design.

Interpretation: what the patterns bought us

+ Benefits

- New behaviour arrives as new classes, not edits — a new observer is one class and one registration
- State removes status if-chains; illegal transitions fail in one place (409)
- Injected clock made tests deterministic

± Trade-offs

- More types and DI registrations to navigate
- Factory has one creator — closer to Simple Factory
- Weekend surcharge applies to the whole rental; seasonal checks pickup month only
- Observers run synchronously inside the request

! Limitations

- Change1 code reconstructed from its documents
- Evolved app built and tested but not run against MySQL
- Change2/4 commit hashes exist only in local clones
- No complexity or coverage figures for C#

04

Part 3 · Cross-language comparison

One representative instance per pattern, same specification and tests in Java, Python, JavaScript and C++

Selected instances and common specification

Factory Method	**Vehicle Category Rules Creation · from Change2** 7 shared test cases — deposits, COMMERCIAL weekly-rate error, unknown category
Observer	**Booking Lifecycle Notification · from Change4** 8 cases — subscribe once, order, unsubscribe, failing observer isolated
Dependency Injection	**DI Composition · original code, extended by Change1** 10 cases — lifetimes, open generics, constructor wiring
Strategy	**Rental Pricing Policy Selection · from Change3** 1 self-checking example — all four print “Weekend: 360.00”
Repository	**Generic Repository CRUD · original code** 1 self-checking example — all four print “Repository CRUD: PASS”

Measurement rules

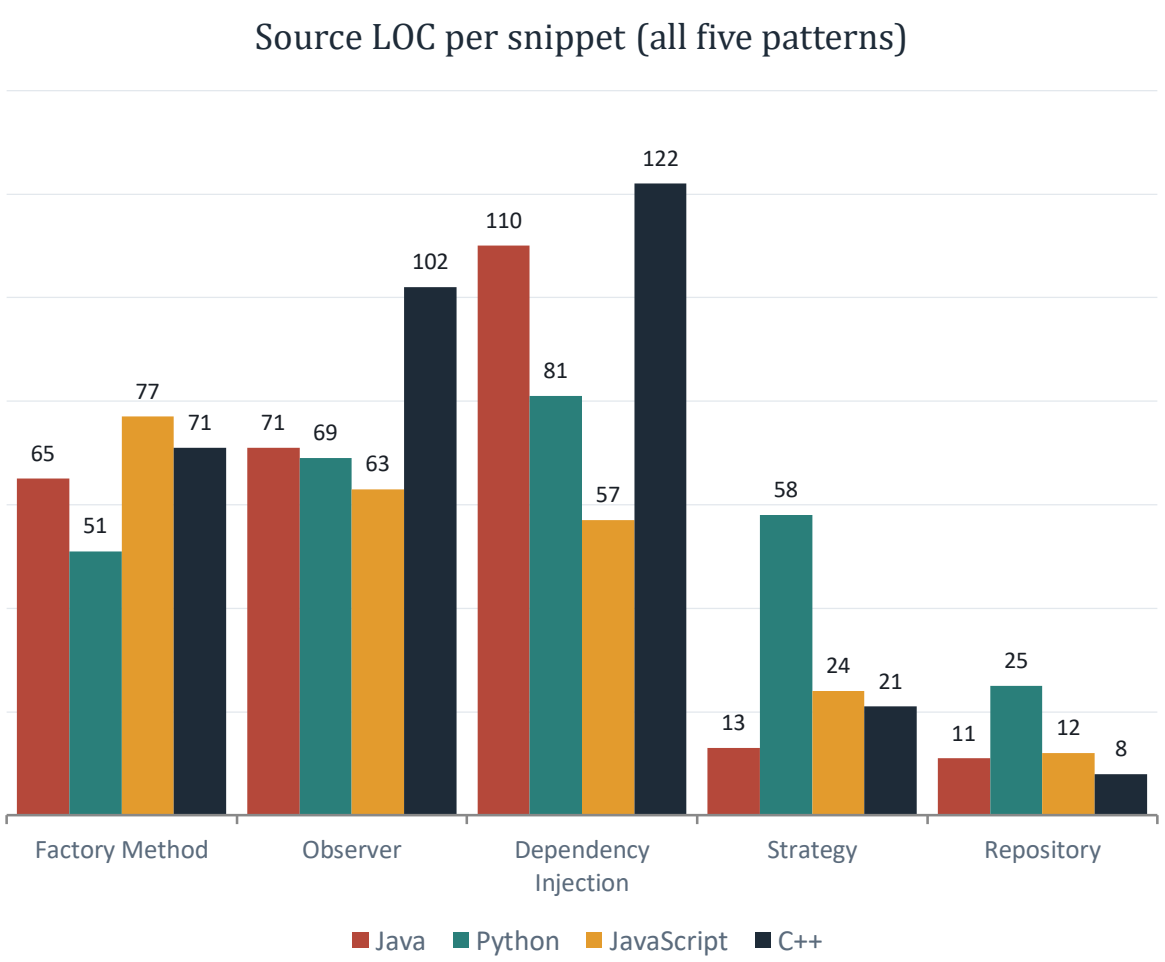
Spec: one common_spec.md per pattern; every port implements exactly it

LOC: snippet file only, no blank or comment lines, tests excluded

Max CC: 1 + each if, loop, case, catch, &&, ||

Time: compile, 1 warm-up, 5 runs, median; whole process, so start-up is included

Results: all ports pass, structure stays the same



Median ms · max CC	Java	Python	JS	C++
Factory Method (7/7)	110 · 4	57 · 2	70 · 4	15 · 4
Observer (8/8)	239 · 9	65 · 3	71 · 3	17 · 6
Dependency Inj. (10/10)	853 · 7	132 · 8	154 · 8	10 · 7
Strategy (1 example)	117 · 5	92 · 5	67 · 5	12 · 5
Repository (1 example)	80 · 6	58 · 2	70 · 2	13 · 7

All 20 ports (5 patterns × 4 languages) pass.

- Times include process start-up: the JVM dominates Java, native C++ starts in ~10–17 ms.
- Measured on different machines: compare within a row, not across rows.
- Strategy/Repository Java, JS and C++ files put several statements per line, so their low LOC is not comparable.

How each language expresses the same roles

Java

- Nominal interfaces checked at compile time
- Exhaustive switch over an enum — a missing category is a compile error
- Single-method observer interface, so lambdas work as observers
- DI by reflection; type erasure forces string tokens

Python

- abc.ABC enforced only at construction; duck typing still works
- Factory looks the class up in a dict — no switch
- Frozen dataclass as the event
- Classes are callables: impl(*deps)

JavaScript

- No interfaces: base classes throw “not implemented”
- Frozen objects stand in for enums
- #private fields hold state
- EventEmitter avoided — a throwing listener stops the rest

C++

- Pure virtual abstract classes, virtual destructors
- unique_ptr / shared_ptr make ownership explicit
- Variadic templates wire DI at compile time
- enum class needs a hand-written to_string

05

LLM records and human verification

Every LLM-assisted step saved with prompt, input, output and an honest evaluation

What the LLMs did, and what we verified

Part	Model · date	Task	Evaluation
Part 1	GPT-5.6 Luna · 2026-10-08	Pattern discovery on source context	Checked by hand: 9 kept, 3 uncertain and 9 rejected excluded
Part 2	Claude Opus 5.5 · 2026-10-10	Change4 Observer code and evidence	Correct with corrections — 200/201 re-run; commit only local
Part 2	Claude Opus 5.5 · 2026-10-11	Integrate Change1–4 into Part2/code	Correct for what ran: 297/297; not run on MySQL
Part 2	Claude Opus 5.5 · 2026-10-11	Rewrite Change1 (no diff existed)	Cannot confirm as the team's code: 50 test names match, LOC 3956 vs 3957
Part 2	Claude Opus 5.5 · 2026-10-11	Evolution CSV (55 rows), counts, 9 UML diagrams	Correct with limits — role mapping needs human review
Part 3	Claude Opus 5.5 · 2026-10-10	Factory Method and Observer ports	Correct (FM with path corrections) — logs, LOC and timings checked
Part 3	Claude Opus 5.5 · 2026-10-10	Dependency Injection ports	Partial — 10/10 in author layout; not re-run here
Part 3	GPT-6 · 2026-10-11; measured with Claude Opus 5.5	Strategy and Repository ports	Correct for what ran — all 8 exit 0; first CSV rows replaced by measured values

Prompts: Part1/Part1_prompt.txt · Part2/part2_prompt.txt · Part2/changes/Change4/prompt.md · Part3/llm/part3_prompt.txt— records in each part's llm.csv

06

Findings and contributions

Evidence-supported conclusions, who did what, and references

Findings

1

Framework-shaped baseline

All 9 confirmed instances are enterprise, architectural or integration patterns — no behavioural GoF pattern was present.

2

Missing patterns were the extension points

State, Factory Method, Strategy and Observer — rejected or doubtful at baseline — each solved one real requirement.

3

Existing patterns absorbed change

No instance removed or replaced; DI extended by every change, Repository gained clients without edits.

4

Refactor where the smell is real

Two refactorings: static pricing coupling with a hidden clock (C3), and a long mixed-responsibility method (C4).

5

Tests prove behaviour kept

183 → 297 tests, all passing; the one baseline failure was a clock dependency fixed by injection.

6

Roles are language-independent

Four languages, same participants and tests; language features change LOC and CC, not the pattern.

Contributions and team folder

Roll number	Name	Contribution
AM.SC.U4CSE23251	Samridhi Singh	Change1: maintenance scheduling, State pattern, 50 tests
AM.SC.U4CSE23211	Bhavana P H	Change2: vehicle categories, Factory Method, 24 tests
AM.SC.U4CSE23238	Neema Vinod	Change3: pricing policies, Strategy, refactoring, 22 tests
AM.SC.U4CSE23266	Vani Sugovind S R	Change4: booking notifications, Observer, refactoring, 18 tests

Parts 1 and 3, the integration and the report were shared team work.

```
B6/
├ README.md
├ data/
│   ├ code/  original (dfc61ab8)
│   └ test_result/
├ Part1/
│   ├ part1_patterns.csv
│   ├ part1_pattern_count.csv
│   ├ pattern_architecture/
│   └ Part1_prompt.txt, llm.csv
├ Part2/
│   ├ code/  evolved source
│   ├ changes/Change1…4, integration/
│   ├ pattern_evaluation.csv
│   ├ Pattern_count.csv, diagrams/
│   └ part2_prompt.txt, llm.csv
└ Part3/
    ├ Factory_Method/, Observer_Pattern/
    ├ Dependency_Injection_Pattern/
    ├ strategy_pattern/
    ├ repository_pattern/
    └ llm/
```

References

- Application: Naveen-verma-7697/FleetManagment, GitHub, commit dfc61ab8 (accessed Oct 2026)
- Submission: aswathymohan-amrita/23CSE455_DP_CaseStudy, folder B6
- Gamma, Helm, Johnson, Vlissides — Design Patterns: Elements of Reusable Object-Oriented Software, 1994
- Fowler — Patterns of Enterprise Application Architecture, 2002 (Repository, Service Layer, DTO, Unit of Work)
- Fowler — “Strangler Fig Application”, martinfowler.com
- Microsoft Learn — ASP.NET Core dependency injection, middleware, hosted services, System.Threading.Channels, TimeProvider
- Tools: .NET SDK 8.0.425, NUnit 3.14, Moq, FluentAssertions, PlantUML 1.2024.8, lizard
- LLMs: GPT-5.6 Luna (Part 1); Claude Opus 5.5 in Claude Code (Parts 2 and 3); GPT-6 (Part 3 Strategy and Repository)

Thank you

Questions? · Team B6

Repository · Dependency Injection · Service Layer · Middleware Chain · Producer–Consumer · Adapter · Proxy · Strangler Fig · DTO · State · Factory Method · Strategy · Observer